Feedback Performance Evaluation

Răspunsul dvs. a fost înregistrat.

Trimiteți alt răspuns

Acest conținut nu este nici creat, nici aprobat de Google. Raportați un abuz - Condiții de utilizare - Politica de confidențialitate

Formulare Google

EP - Data Manipulation & Advanced Plotting (pandas, seaborn & 3D Plotting)

In this lab, we will study data manipulation and visualization using **pandas**, and explore the high level API of **seaborn** for generating visually appealing plots. We will also take a look at 3D plotting using **mplot3d**.

```
# Some IPython magic
# Put these at the top of every notebook, to get automatic reloading and inline plotting
%reload_ext autoreload
%autoreload 2

import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Check out these cheetsheets for fast reference to the common libraries:

Cheat sheets:

- python
- numpy
- matplotlib
- sklearn
- pandas

Other:

- Probabilities & Stats Refresher
- Algebra

Pandas Crash Course

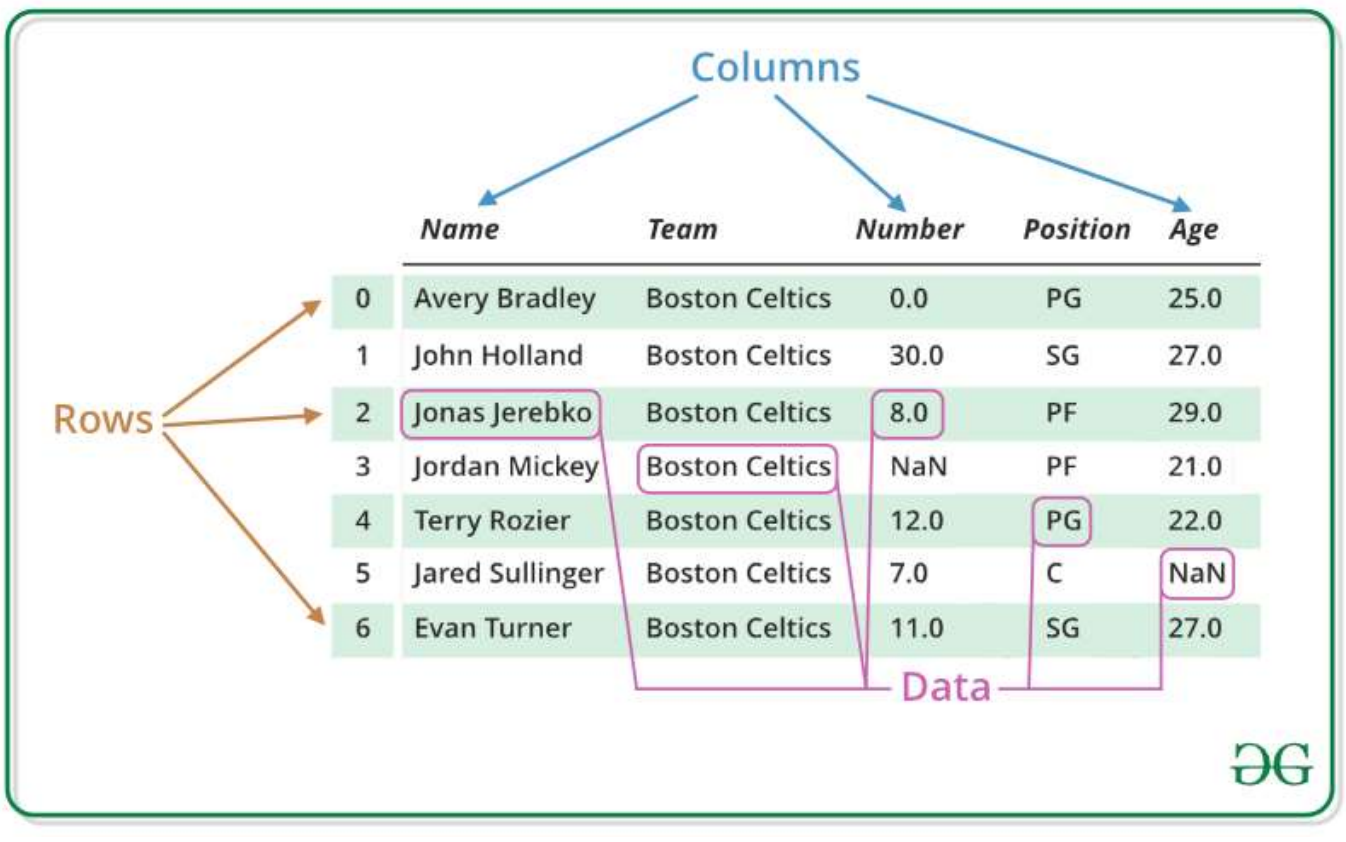
Pandas is a high-level data manipulation tool. It is built on the Numpy package and its key data structure is called the DataFrame. DataFrames allow you to store and manipulate tabular data in rows of observations and columns of variables.

Check this official guide for a started in pandas:

10 minutes to pandas

```
import pandas as pd
```

Pandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.



Let's load a publicly available *.csv* dataset into a pandas ***DataFrame***. We will use the popular *iris* dataset.

```
file_name = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"

df = pd.read_csv(file_name)
df.head(n = 10)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
6	4.6	3.4	1.4	0.3	setosa
7	5.0	3.4	1.5	0.2	setosa
8	4.4	2.9	1.4	0.2	setosa
9	4.9	3.1	1.5	0.1	setosa

A dataframe's **.describe()** method offers descriptive statistics which include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding NaN values.

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Let's see some ways in which we can access the DataFrames' data. Each column of a pandas `DataFrame` is a pandas `Series`.

```
df['petal_width']
```

```
0      0.2
1      0.2
2      0.2
3      0.2
4      0.2
      ...
145    2.3
146    1.9
147    2.0
148    2.3
149    1.8
Name: petal_width, Length: 150, dtype: float64
```

We can do any vectorized operation on a `Series`. Moreover, a pandas `Series` allows us to do conditional selection of rows in a `DataFrame`.

```
setosas = df[df['species'] == 'setosa']

setosas.head() # only setosa species selected
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

We can add a new column to a pandas `DataFrame`, simply by specifying its name and its contents.

NB: the data added to the new column must be the same length as the rest of the `DataFrame`.

```
df['sepal_area'] = df['sepal_length'] * df['sepal_width'] # adding new columns
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	5.1	3.5	1.4	0.2	setosa	17.85

We can work with `Series` as we work with numpy arrays. We perform Min-Max normalization on the `petal_length` column.

```
# Min-Max Normalization
df['petal_length'] = (df['petal_length'] - df['petal_length'].min()) / (df['petal_length'].max() - df['petal_length'].min())
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	5.1	3.5	0.067797	0.2	setosa	17.85
1	4.9	3.0	0.067797	0.2	setosa	14.70
2	4.7	3.2	0.050847	0.2	setosa	15.04
3	4.6	3.1	0.084746	0.2	setosa	14.26
4	5.0	3.6	0.067797	0.2	setosa	18.00

We can also use the `.apply()` method on either a `Series` or a `DataFrame` to modify its contents, or create a new column.

```
def capitalize(col):
  return col.capitalize()

df['species'] = df['species'].apply(capitalize)
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	5.1	3.5	0.067797	0.2	Setosa	17.85
1	4.9	3.0	0.067797	0.2	Setosa	14.70
2	4.7	3.2	0.050847	0.2	Setosa	15.04
3	4.6	3.1	0.084746	0.2	Setosa	14.26
4	5.0	3.6	0.067797	0.2	Setosa	18.00

A `DataFrame` also has a `groupby` method, that allows us to work on groupings of rows.

```
df.groupby('species').mean()
```

species	sepal_length	sepal_width	petal_length	petal_width	sepal_area
Setosa	5.006	3.418	0.078644	0.244	17.2088
Versicolor	5.936	2.770	0.552542	1.326	16.5262
Virginica	6.588	2.974	0.771525	2.026	19.6846

We can also iterate through each group. A group is another `DataFrame`.

```
for name, group in df.groupby('species'):
  print("Group:", name)
  print(group.head())
  print("-----")
```

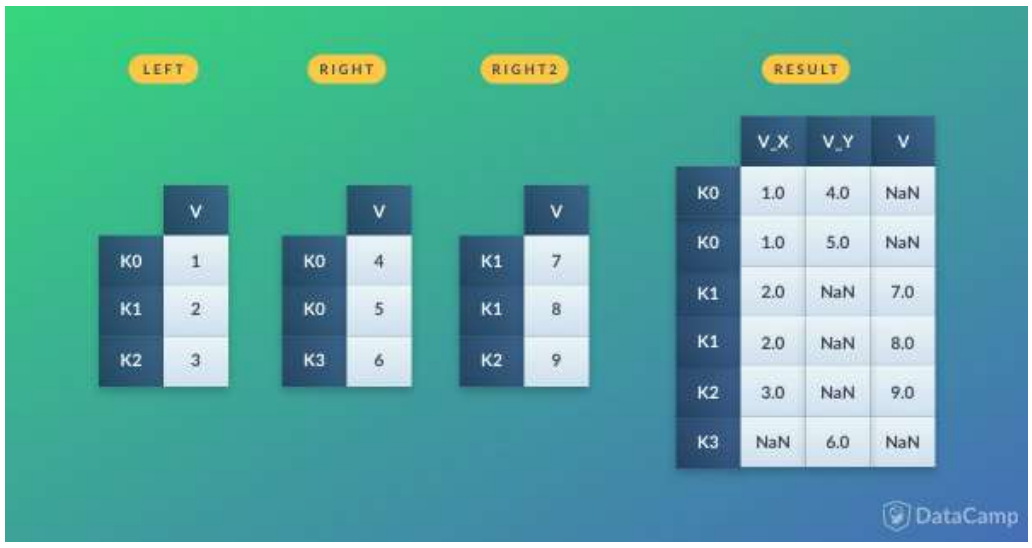
```
Group: Setosa
   sepal_length  sepal_width  petal_length  petal_width species  sepal_area
0           5.1          3.5      0.067797          0.2  Setosa       17.85
1           4.9          3.0      0.067797          0.2  Setosa       14.70
2           4.7          3.2      0.050847          0.2  Setosa       15.04
3           4.6          3.1      0.084746          0.2  Setosa       14.26
4           5.0          3.6      0.067797          0.2  Setosa       18.00
-----
Group: Versicolor
    sepal_length  sepal_width  ...     species  sepal_area
50           7.0          3.2  ...  Versicolor       22.40
51           6.4          3.2  ...  Versicolor       20.48
52           6.9          3.1  ...  Versicolor       21.39
53           5.5          2.3  ...  Versicolor       12.65
54           6.5          2.8  ...  Versicolor       18.20

[5 rows x 6 columns]
-----
Group: Virginica
     sepal_length  sepal_width  ...    species  sepal_area
100           6.3          3.3  ...  Virginica       20.79
101           5.8          2.7  ...  Virginica       15.66
102           7.1          3.0  ...  Virginica       21.30
103           6.3          2.9  ...  Virginica       18.27
104           6.5          3.0  ...  Virginica       19.50

[5 rows x 6 columns]
-----
```

Joins

Pandas allows for joining two or more `DataFrames` together using a common key. We can also do vertical or horizontal concatenation .




```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']})
df2 = pd.DataFrame({'A': ['A0', 'A1', 'A4', 'A5'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']})
df3 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],
                    'B': ['B8', 'B9', 'B10', 'B11'],
                    'C': ['C8', 'C9', 'C10', 'C11'],
                    'D': ['D8', 'D9', 'D10', 'D11']})
```

```
pd.concat([df1, df2, df3]).reset_index(drop = True)
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A0	B4	C4	D4
5	A1	B5	C5	D5
6	A4	B6	C6	D6
7	A5	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

```
pd.merge(df1, df2, on = 'A', how = 'left')
```

	A	B_x	C_x	D_x	B_y	C_y	D_y
0	A0	B0	C0	D0	B4	C4	D4
1	A1	B1	C1	D1	B5	C5	D5
2	A2	B2	C2	D2	NaN	NaN	NaN
3	A3	B3	C3	D3	NaN	NaN	NaN

Saving `DataFrames`

Pandas offers a multitude of methods for saving `DataFrames` .

```
df.to_csv('out.csv', index = False) # saves it locally, check out the files in the right tab of colab
```

```
df.to_json()
```

```
'{"sepal_length":{"0":5.1,"1":4.9,"2":4.7,"3":4.6,"4":5.0,"5":5.4,"6":4.6,"7":5.0,"8":4.4,"9":4.9,"10":5.4,"11":4.8,"12":4.8,"13":4.3,"14":5.8,"15":5.
7,"16":5.4,"17":5.1,"18":5.7,"19":5.1,"20":5.4,"21":5.1,"22":4.6,"23":5.1,"24":4.8,"25":5.0,"26":5.0,"27":5.2,"28":5.2,"29":4.7,"30":4.8,"31":5.4,"3
2":5.2,"33":5.5,"34":4.9,"35":5.0,"36":5.5,"37":4.9,"38":4.4,"39":5.1,"40":5.0,"41":4.5,"42":4.4,"43":5.0,"44":5.1,"45":4.8,"46":5.1,"47":4.6,"48":5.
3,"49":5.0,"50":7.0,"51":6.4,"52":6.9,"53":5.5,"54":6.5,"55":5.7,"56":6.3,"57":4.9,"58":6.6,"59":5.2,"60":5.0,"61":5.9,"62":6.0,"63":6.1,"64":5.6,"6
5":6.7,"66":5.6,"67":5.8,"68":6.2,"69":5.6,"70":5.9,"71":6.1,"72":6.3,"73":6.1,"74":6.4,"75":6.6,"76":6.8,"77":6.7,"78":6.0,"79":5.7,"80":5.5,"81":5.
5,"82":5.8,"83":6.0,"84":5.4,"85":6.0,"86":6.7,"87":6.3,"88":5.6,"89":5.5,"90":5.5,"91":6.1,"92":5.8,"93":5.0,"94":5.6,"95":5.7,"96":5.7,"97":6.2,"9
8":5.1,"99":5.7,"100":6.3,"101":5.8,"102":7.1,"103":6.3,"104":6.5,"105":7.6,"106":4.9,"107":7.3,"108":6.7,"10…'
```

```
print(df.head().to_markdown())
```

```
|    |   sepal_length |   sepal_width |   petal_length |   petal_width | species   |   sepal_area |
|---:|---------------:|--------------:|---------------:|--------------:|:----------|-------------:|
```

```
|  0 |           5.1 |           3.5 |      0.0677966 |           0.2 | Setosa    |        17.85 |
|  1 |           4.9 |           3   |      0.0677966 |           0.2 | Setosa    |        14.7  |
|  2 |           4.7 |           3.2 |      0.0508475 |           0.2 | Setosa    |        15.04 |
|  3 |           4.6 |           3.1 |      0.0847458 |           0.2 | Setosa    |        14.26 |
|  4 |           5   |           3.6 |      0.0677966 |           0.2 | Setosa    |        18    |
```

```
print(df.head().to_latex())
```

```
\begin{tabular}{lrrrrlr}
\toprule
{} &  sepal\_length &  sepal\_width &  petal\_length &  petal\_width & species &  sepal\_area \\
\midrule
0 &          5.1 &         3.5 &      0.067797 &          0.2 &  Setosa &       17.85 \\
1 &          4.9 &         3.0 &      0.067797 &          0.2 &  Setosa &       14.70 \\
2 &          4.7 &         3.2 &      0.050847 &          0.2 &  Setosa &       15.04 \\
3 &          4.6 &         3.1 &      0.084746 &          0.2 &  Setosa &       14.26 \\
4 &          5.0 &         3.6 &      0.067797 &          0.2 &  Setosa &       18.00 \\
\bottomrule
\end{tabular}
```

```
print(df.head(n = 3).to_html())
```

```
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>sepal_length</th>
      <th>sepal_width</th>
      <th>petal_length</th>
      <th>petal_width</th>
      <th>species</th>
      <th>sepal_area</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>5.1</td>
      <td>3.5</td>
      <td>0.067797</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>17.85</td>
    </tr>
    <tr>
      <th>1</th>
      <td>4.9</td>
      <td>3.0</td>
      <td>0.067797</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>14.70</td>
    </tr>
    <tr>
      <th>2</th>
      <td>4.7</td>
      <td>3.2</td>
      <td>0.050847</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>15.04</td>
    </tr>
  </tbody>
</table>
```

```
df.to_sql(name = '<table_name>', con = '<connection>') # insert into a sql database, works with a valid connection
# check out https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_sql.html
```

```
---------------------------------------------------------------------------
ArgumentError                             Traceback (most recent call last)
<ipython-input-62-6d777a02bc3c> in <module>()
----> 1 df.to_sql(name = '<table_name>', con = '<connection>') # insert into a sql database, works with a valid connection
      2 # check out https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_sql.html

 7 frames
/usr/local/lib/python3.6/dist-packages/sqlalchemy/engine/url.py in _parse_rfc1738_args(name)
    289     else:
    290         raise exc.ArgumentError(
--> 291             "Could not parse rfc1738 URL from string '%s'" % name
    292         )
    293

ArgumentError: Could not parse rfc1738 URL from string '<connection>'
```

SEARCH STACK OVERFLOW

Pandas Plotting

Pandas offers a convenient API for plotting data directly from a DataFrame. Of course, the plotting API is build upon `matplotlib` as a low level backend. We can use that to manipulate plots as in the previous lab. Check out the official documentation for visualization:

Pandas Plotting Docs

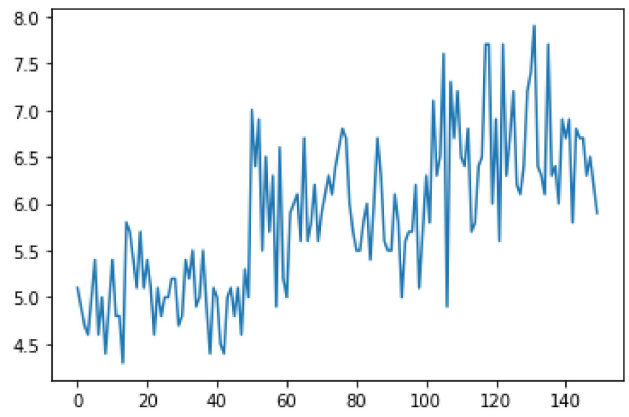
For a quick reference, check the official cookbook.

Pandas Plotting Cookbook

We will use the high level plotting API to visualize the Iris Dataset.

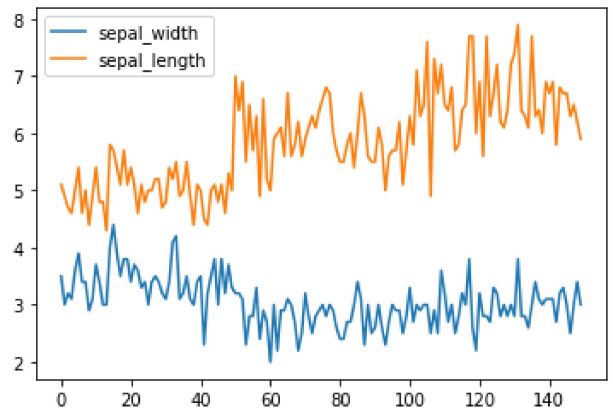
```
df['sepal_length'].plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd73f1b9160>
```




```
df[['sepal_width', 'sepal_length']].plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd73f126630>
```




```
fig, ax = plt.subplots(1, 2)

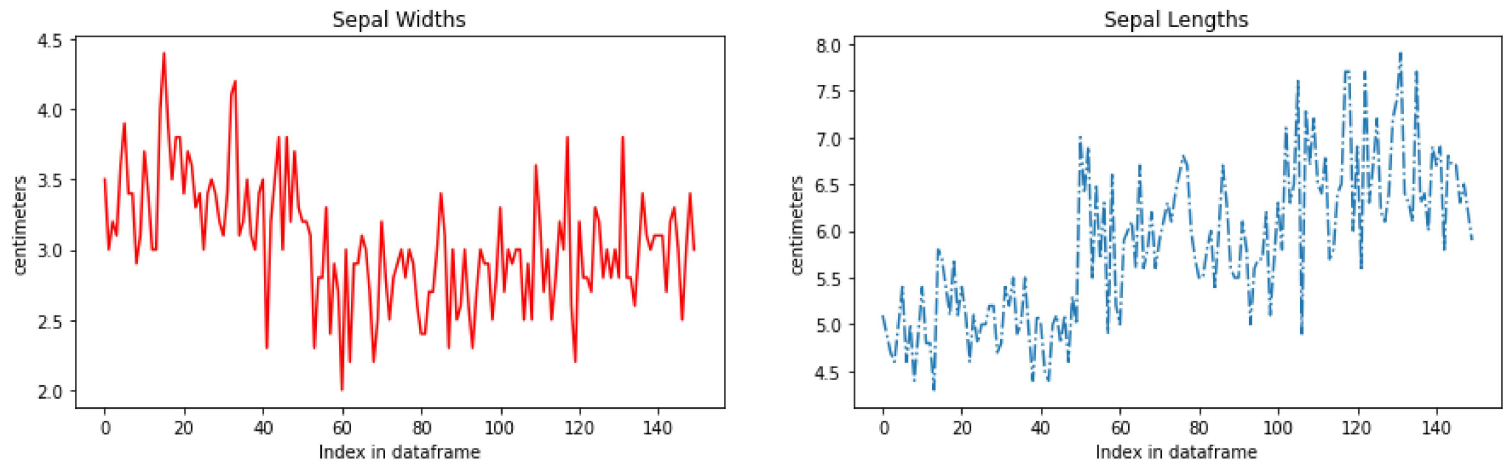
df['sepal_width'].plot(ax = ax[0], color = 'r')
df['sepal_length'].plot(ax = ax[1], linestyle = '-.')

ax[0].set_title('Sepal Widths')
ax[1].set_title('Sepal Lengths')

ax[0].set_xlabel('Index in dataframe')
ax[1].set_xlabel('Index in dataframe')

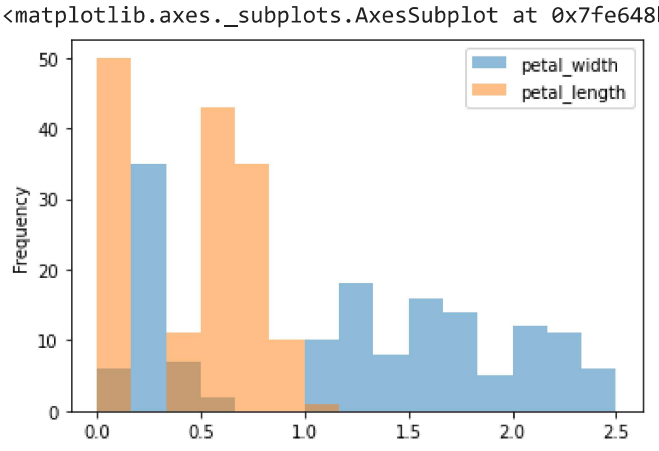
ax[0].set_ylabel('centimeters')
ax[1].set_ylabel('centimeters')


fig.set_size_inches(15, 4)
```



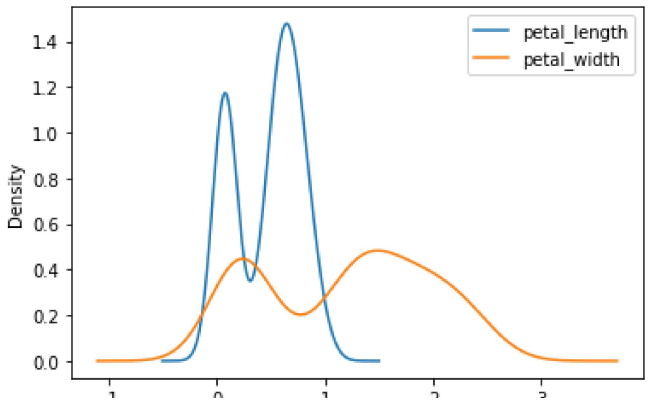

```
df[['petal_width', 'petal_length']].plot.hist(alpha = 0.5, bins = 15)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fe648bce278>
```




```
df[['petal_length', 'petal_width']].plot.kde()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fe648ab5748>
```



Cool Plots using Seaborn & Pandas

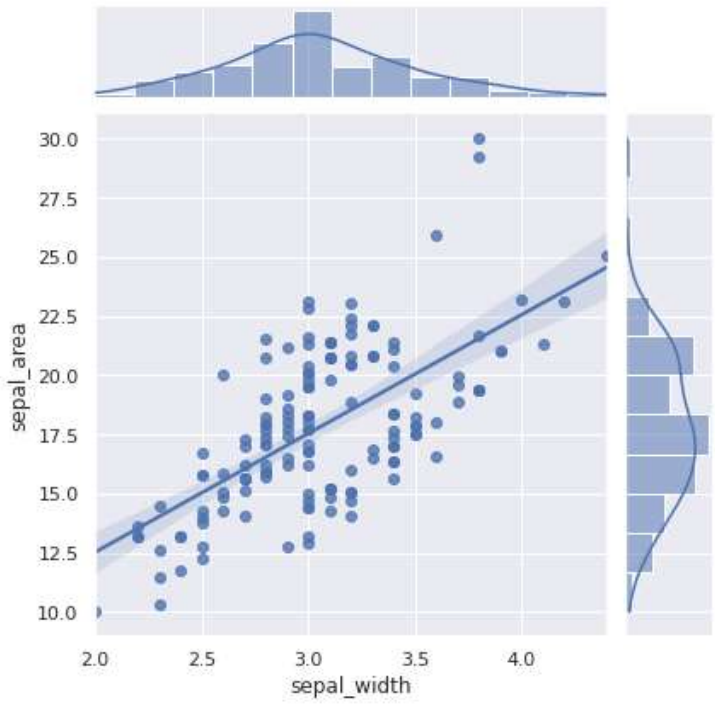
Check out seaborn for more awesome plots.

```
import seaborn as sns

sns.set_theme()
```

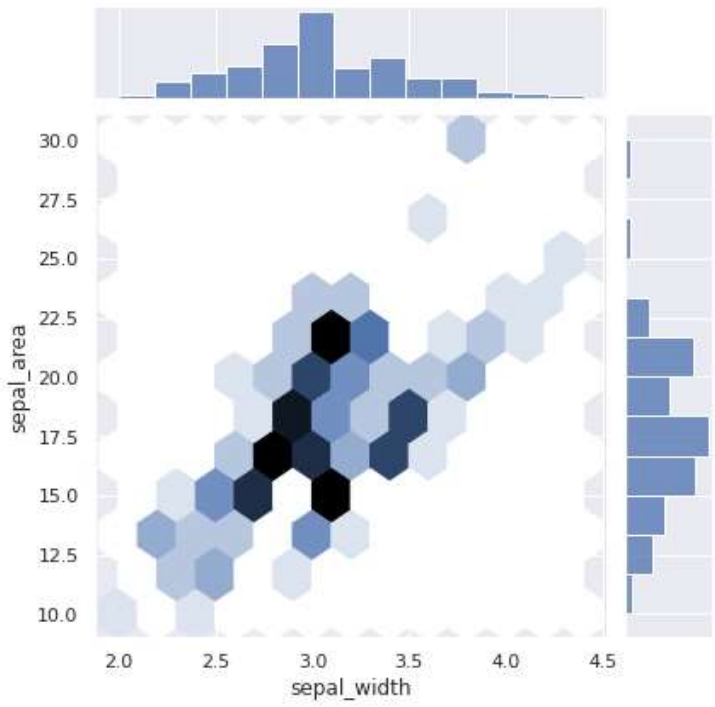
```
sns.jointplot(x = 'sepal_width', y = 'sepal_area', data = df, kind = 'reg')
```

```
<seaborn.axisgrid.JointGrid at 0x7fe64892e940>
```



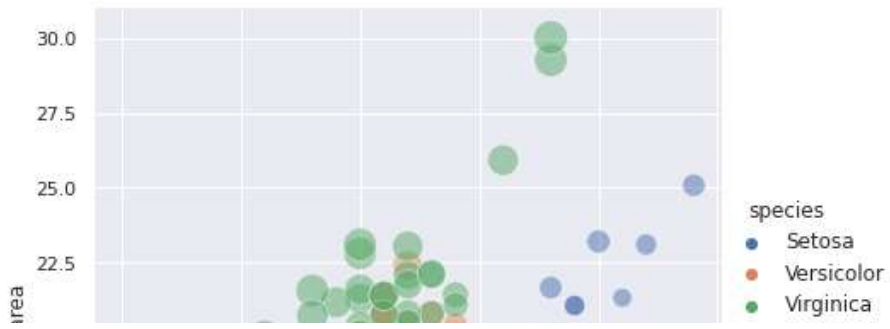

```
sns.jointplot(x = 'sepal_width', y = 'sepal_area', data = df, kind = 'hex')
```

```
<seaborn.axisgrid.JointGrid at 0x7fe63c004828>
```

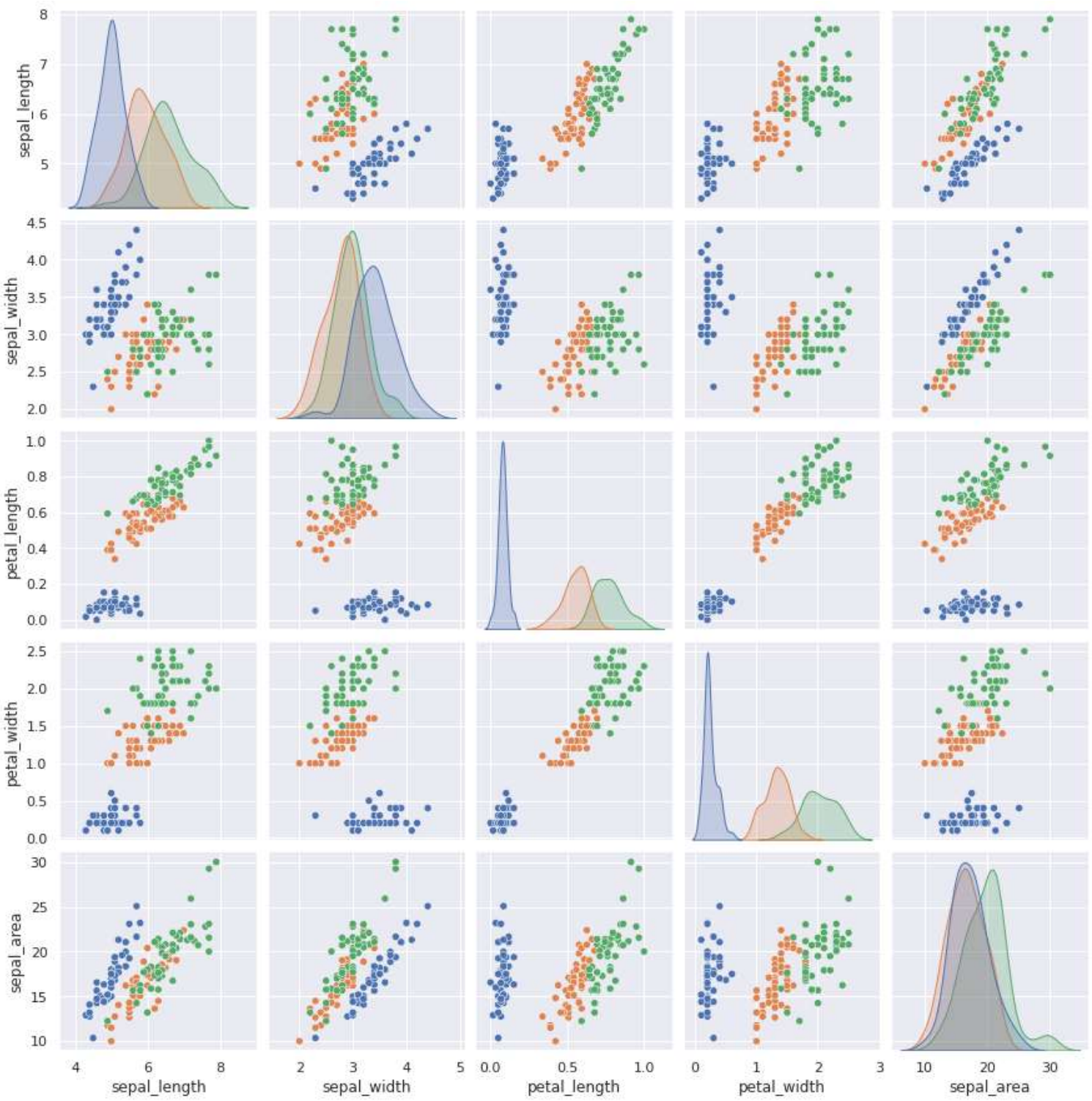



```
sns.relplot(x="sepal_width", y="sepal_area", hue="species", size="sepal_length", sizes=(40, 400), alpha=.5, height=6, data=df)
```

```
<seaborn.axisgrid.FacetGrid at 0x7fe63982bfd0>
```

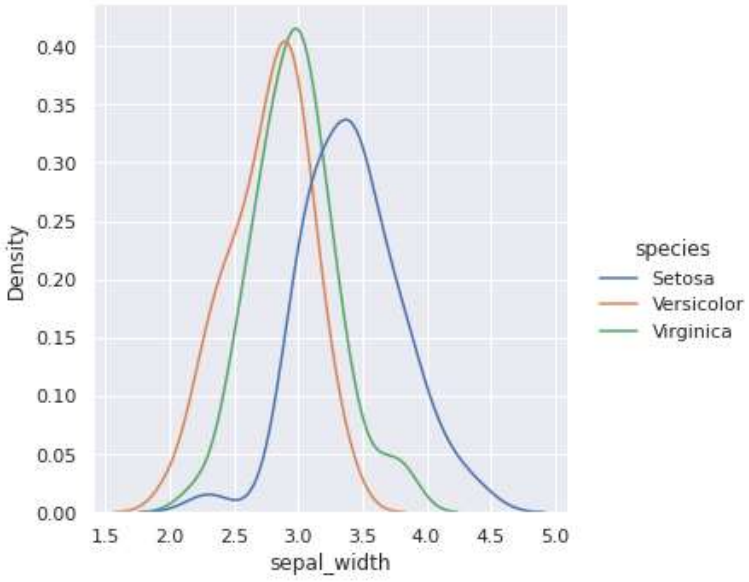



```
grid = sns.pairplot(df, hue = 'species')
```



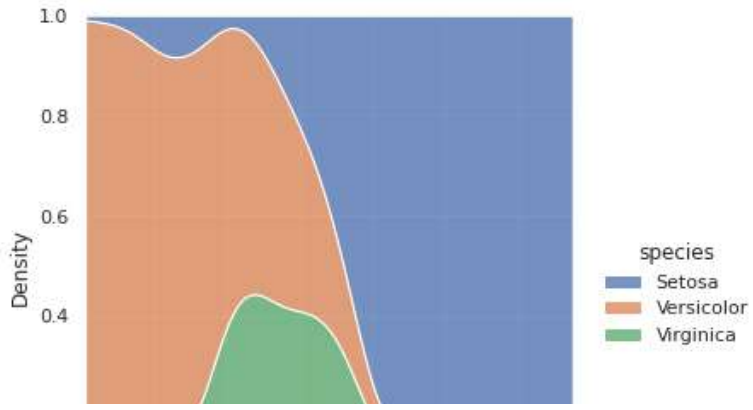

```
sns.displot(data = df, x = 'sepal_width', kind = 'kde', hue = 'species')
```

```
<seaborn.axisgrid.FacetGrid at 0x7fe63b4fbc50>
```



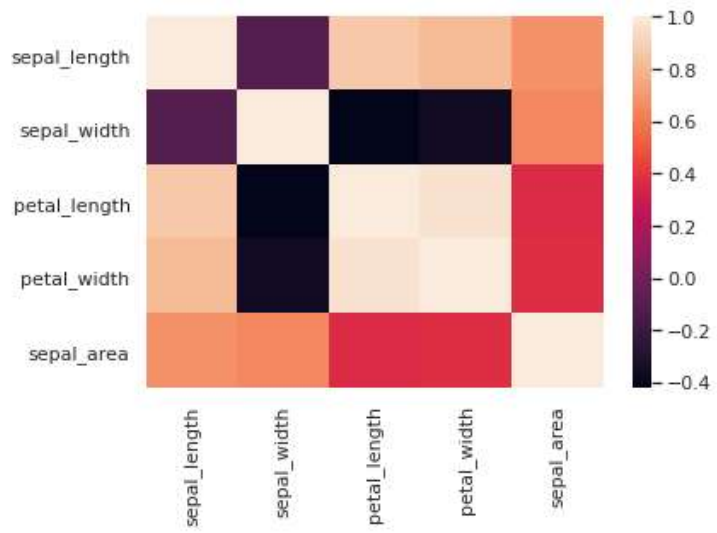

```
sns.displot(data = df, x = 'sepal_width', kind = 'kde', hue = 'species', multiple = 'fill')
```

```
<seaborn.axisgrid.FacetGrid at 0x7fe63b50fe48>
```



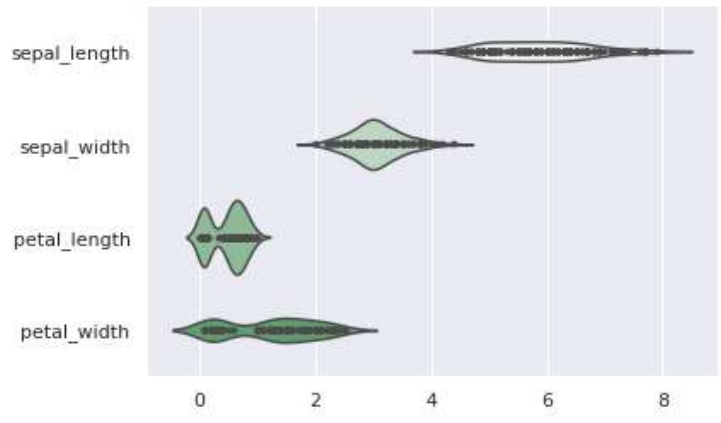

```
sns.heatmap(df.corr())
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd739b23e80>
```




```
sns.violinplot(data=df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']], palette="light:g", inner="points", orient="h")
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd7396e0cc0>
```

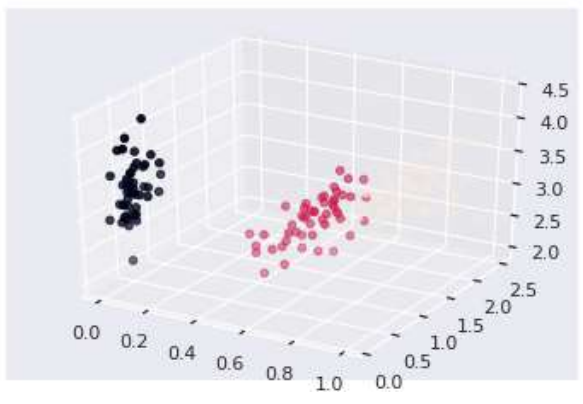


3D Plotting

```
from mpl_toolkits.mplot3d import Axes3D


fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(df['petal_length'], df['petal_width'], df['sepal_width'], zdir='z', s=20, c=df['species'].astype('category').cat.codes, depthshade=Tr
```

```
<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x7fe6396f3898>
```



Exercises

After you finish your lab exercises, you should export this notebook as **pdf** and upload it to Moodle. (i.e. **File -> Print**, Destintation: Save as PDF).

In this lab, we will look at COVID data for Romania. The data was retrieved from https://graphs.ro/.

```
import requests
import json

covid_data = json.loads(requests.get('https://www.graphs.ro/json.php').content)
```

```
covid_df = pd.DataFrame(covid_data['covid_romania'])
covid_df.head()

covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # some days have no county information
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue

  county_df['reporting_date'] = row['reporting_date']
  covid_county_data_dfs.append(county_df)


county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis = 1)


covid_df.head()
```

ths_today	total_recovered	new_recovered_today	intensive_care_right_now	emergency_calls	information_calls	persons_in_quarantine	persons_in_home_iso
186.0	253244.0	6611.0	1174.0	6412.0	1046.0	NaN	4
149.0	246633.0	2801.0	1187.0	3522.0	NaN	NaN	4
113.0	243832.0	4781.0	1169.0	4034.0	NaN	NaN	4
129.0	239051.0	7243.0	1172.0	5800.0	935.0	NaN	4
174.0	231808.0	6892.0	1149.0	4556.0	831.0	NaN	4

```
county_df.head()
```

	county_id	county_name	county_population	total_cases	reporting_date
0	AB	Alba	323778	8050	2020-11-17
1	AR	Arad	415732	8967	2020-11-17
2	AG	Arges	574920	10279	2020-11-17
3	BC	Bacau	580912	10840	2020-11-17
4	BH	Bihor	559992	11269	2020-11-17

1. Basic Visualizations

Make 4 subplots. Using pandas as seaborn, plot the number of new cases in a day, the number of recovered patients in day, number of tests in a day, and the number of deaths in day. We are trying to explore the evolution of COVID from the start of the pandemic until today.

NB: Make sure to add proper labels, title, axes and legend where necessary.

```
import seaborn as sns
import requests
import json
import pandas as pd

covid_data = json.loads(requests.get('https://www.graphs.ro/json.php').content)

covid_df = pd.DataFrame(covid_data['covid_romania'])
covid_df.head()

covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # some days have no county information
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue

  county_df['reporting_date'] = row['reporting_date']
  covid_county_data_dfs.append(county_df)


county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis = 1)
covid_df = covid_df.iloc[::-1]


sns.set_theme()

sns.jointplot(x = 'reporting_date', y = 'new_cases_today', data = covid_df)

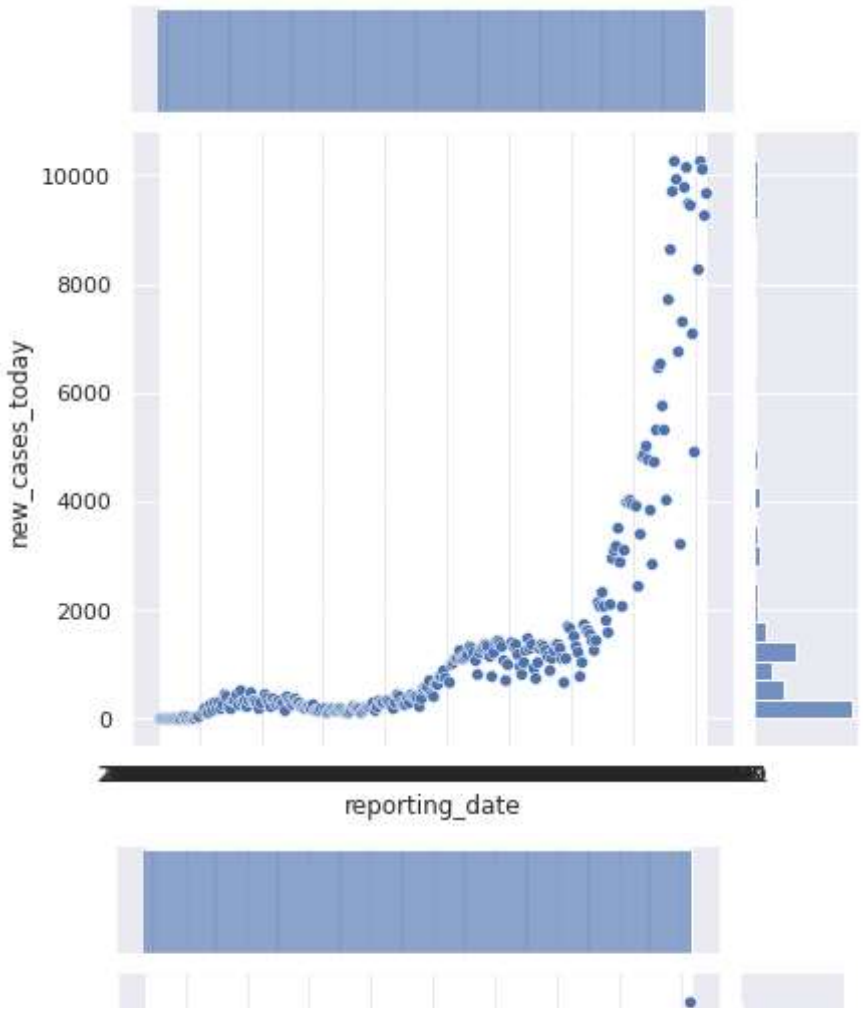
sns.jointplot(x = 'reporting_date', y = 'new_recovered_today', data = covid_df)

sns.jointplot(x = 'reporting_date', y = 'new_tests_today', data = covid_df)

sns.jointplot(x = 'reporting_date', y = 'new_deaths_today', data = covid_df)
```

```
<seaborn.axisgrid.JointGrid at 0x7fdbb4b609b0>
```

```
<seaborn.axisgrid.JointGrid at 0x7fdbb4b609b0>
```



2. Positive testing percentage

Using `pandas`, create a new column that computes the percentage of positive tests in a given day. This new column should be the number of infected people in a day over the number of tests per day.

Plot the evolution of positive tests across time. Compare this to the number of hospitalized patients.

In a different plot, visualize the correlation between positive tests and the number of intensive care patients.

NB: Make sure to add proper labels, title, axes and legend where necessary.

```
import seaborn as sns
import requests
import json
import pandas as pd

covid_data = json.loads(requests.get('https://www.graphs.ro/json.php').content)

covid_df = pd.DataFrame(covid_data['covid_romania'])
covid_df.head()

covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # some days have no county information
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue

  county_df['reporting_date'] = row['reporting_date']
  covid_county_data_dfs.append(county_df)


county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis = 1)
covid_df['percentage'] = (covid_df['new_cases_today']/covid_df['new_tests_today'])
print (covid_df.head())
covid_df = covid_df.iloc[::-1]

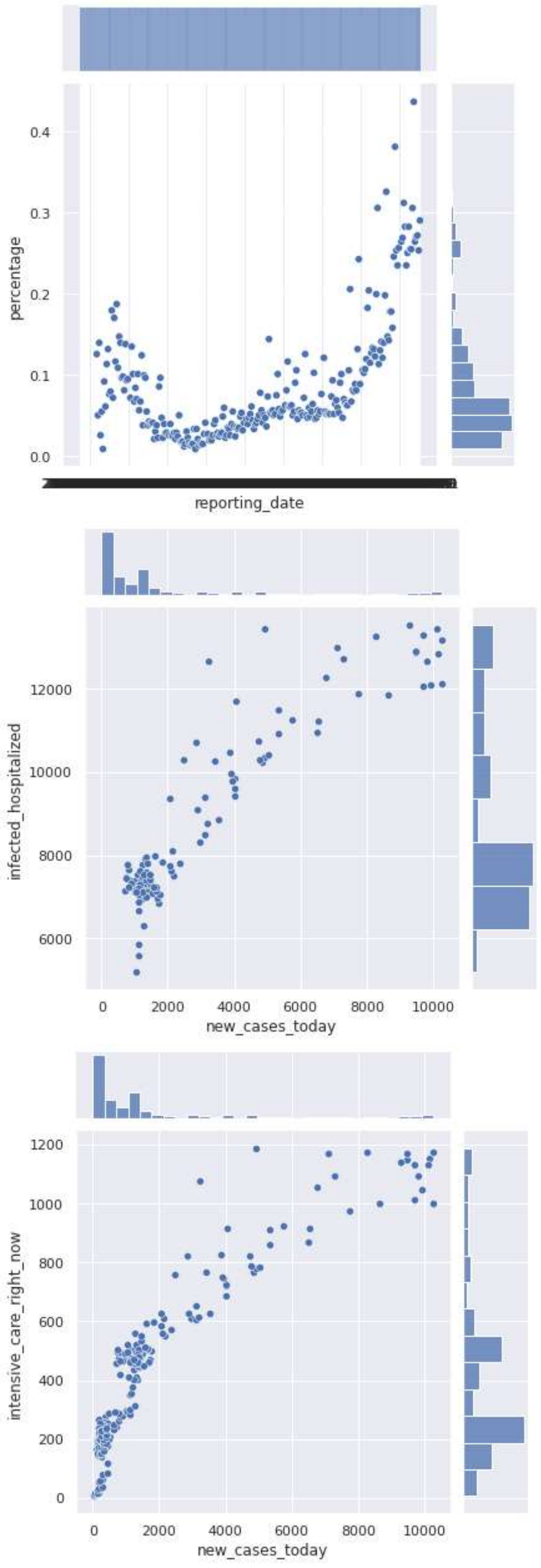
sns.set_theme()


sns.jointplot(x = 'reporting_date', y = 'percentage', data = reversed_covid_df)

sns.jointplot(x = 'new_cases_today', y = 'infected_hospitalized', data = reversed_covid_df)
sns.jointplot(x = 'new_cases_today', y = 'intensive_care_right_now', data = reversed_covid_df)
```

```
  reporting_date  total_cases  ...  romania_population_2020  percentage
0     2020-11-21       412808  ...                 19317984    0.291532
1     2020-11-20       403123  ...                 19317984    0.254355
2     2020-11-19       393851  ...                 19317984    0.273463
3     2020-11-18       383743  ...                 19317984    0.270907
4     2020-11-17       373474  ...                 19317984    0.265813

[5 rows x 25 columns]
<seaborn.axisgrid.JointGrid at 0x7fdbad2bd2e8>
```



3. County Information

Select at least 10 counties of your choosing, and plot the evolution of cases across time.

Plot the number of positive cases on 1000 persons for each of your selected counties. Plot a horizontal line at the 3 / 1000 mark. When the number of positive cases per 1000 persons exceeds 3 / 1000, color your points in a different color from that point onwards.

NB: Make sure to add proper labels, title, axes and legend where necessary.

```
import seaborn as sns
import requests
import json
import pandas as pd

covid_data = json.loads(requests.get('https://www.graphs.ro/json.php').content)

covid_df = pd.DataFrame(covid_data['covid_romania'])
covid_df.head()

covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # some days have no county information
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue
  county_df['reporting_date'] = row['reporting_date']
  covid_county_data_dfs.append(county_df)


county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis = 1)
covid_df = covid_df.iloc[::-1]

covid_df_alba = county_df.loc[county_df['county_name'] == 'Alba']
covid_df_alba = covid_df_alba.iloc[::-1]
covid_df_alba['xx'] = ( ((covid_df_alba['total_cases']/covid_df_alba['county_population']) > 0.003 ) )
covid_df_alba["reporting_date"] = pd.to_datetime(covid_df_alba["reporting_date"])

covid_df_bacau = county_df.loc[county_df['county_name'] == 'Bacau']
covid_df_bacau = covid_df_bacau.iloc[::-1]
covid_df_bacau['category'] = ( ((covid_df_bacau['total_cases']/covid_df_bacau['county_population']) > 0.003 ) )
covid_df_bacau["reporting_date"] = pd.to_datetime(covid_df_bacau["reporting_date"])

covid_df_brasov = county_df.loc[county_df['county_name'] == 'Brasov']
covid_df_brasov = covid_df_brasov.iloc[::-1]
covid_df_brasov['category'] = ( ((covid_df_brasov['total_cases']/covid_df_brasov['county_population']) > 0.003 ) )
covid_df_brasov["reporting_date"] = pd.to_datetime(covid_df_brasov["reporting_date"])

covid_df_galati = county_df.loc[county_df['county_name'] == 'Galati']
covid_df_galati = covid_df_galati.iloc[::-1]
covid_df_galati['category'] = ( ((covid_df_galati['total_cases']/covid_df_galati['county_population']) > 0.003 ) )
covid_df_galati["reporting_date"] = pd.to_datetime(covid_df_galati["reporting_date"])

covid_df_prahova = county_df.loc[county_df['county_name'] == 'Prahova']
covid_df_prahova = covid_df_prahova.iloc[::-1]
covid_df_prahova['category'] = ( ((covid_df_prahova['total_cases']/covid_df_prahova['county_population']) > 0.003 ) )
covid_df_prahova["reporting_date"] = pd.to_datetime(covid_df_prahova["reporting_date"])

sns.set_theme()

sns.jointplot(x = 'reporting_date', y = 'total_cases', data = covid_df_alba, hue='xx')

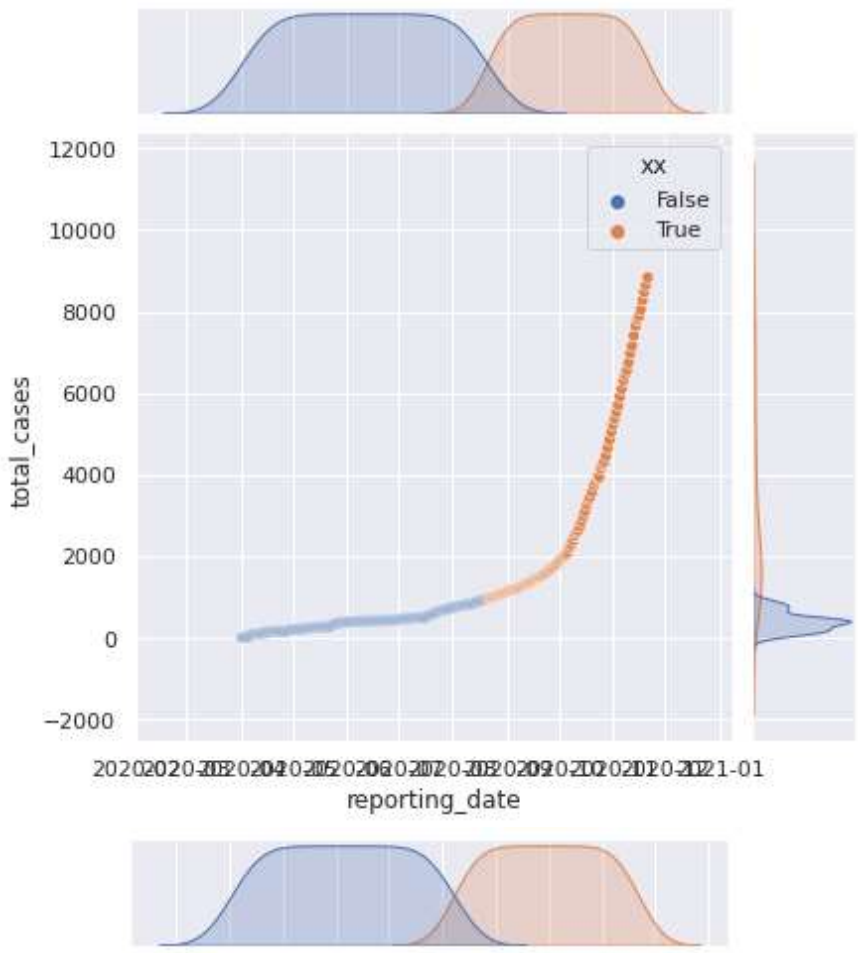
sns.jointplot(x = 'reporting_date', y = 'total_cases', data = covid_df_bacau, hue ='category')

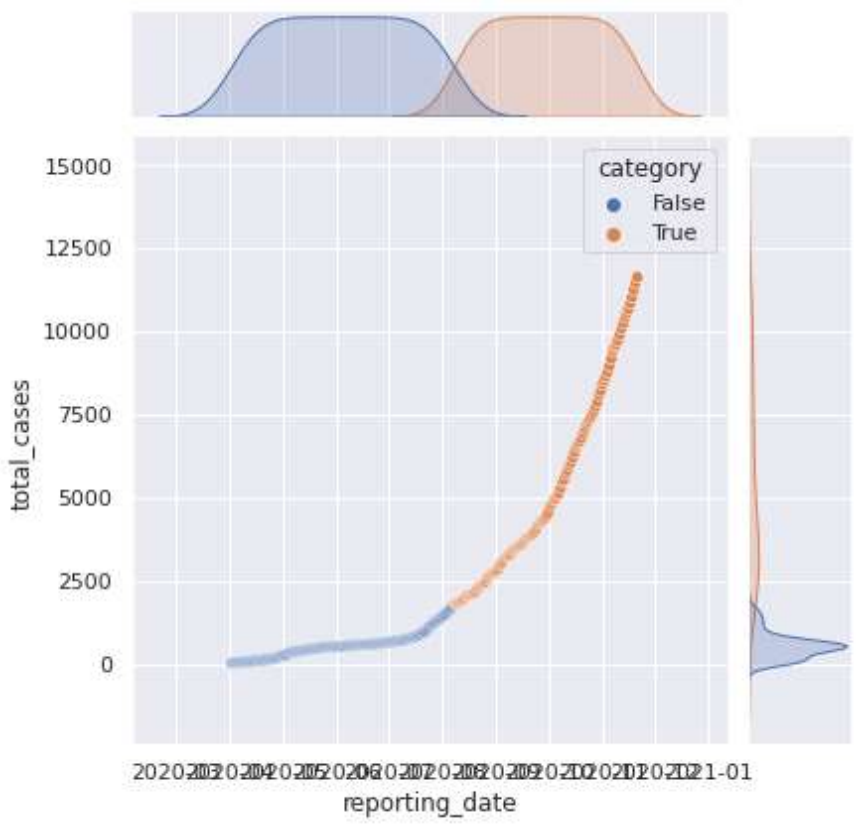
sns.jointplot(x = 'reporting_date', y = 'total_cases', data = covid_df_brasov, hue ='category')

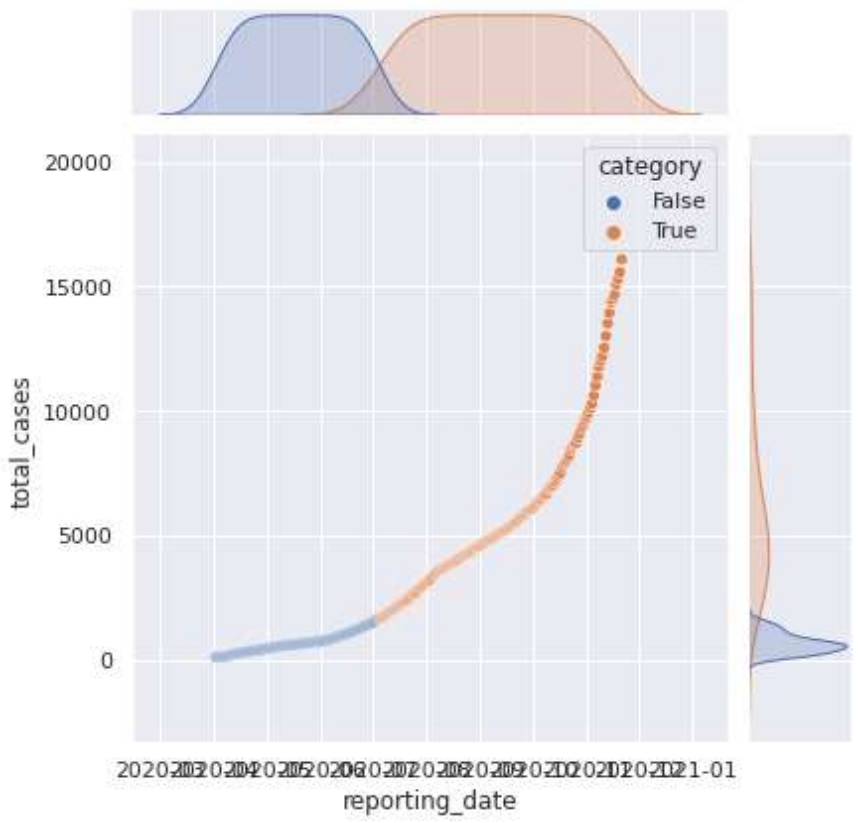
sns.jointplot(x = 'reporting_date', y = 'total_cases', data = covid_df_galati, hue ='category')

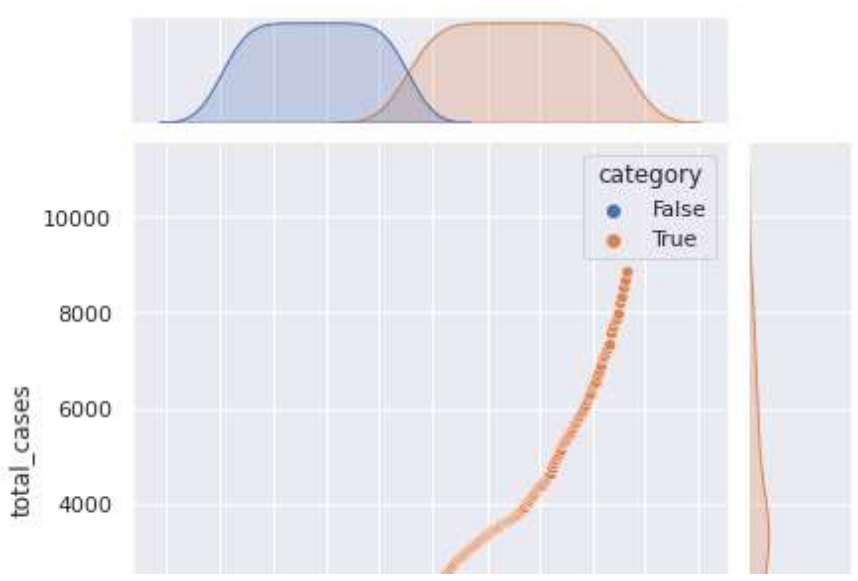
sns.jointplot(x = 'reporting_date', y = 'total_cases', data = covid_df_prahova, hue ='category')
```

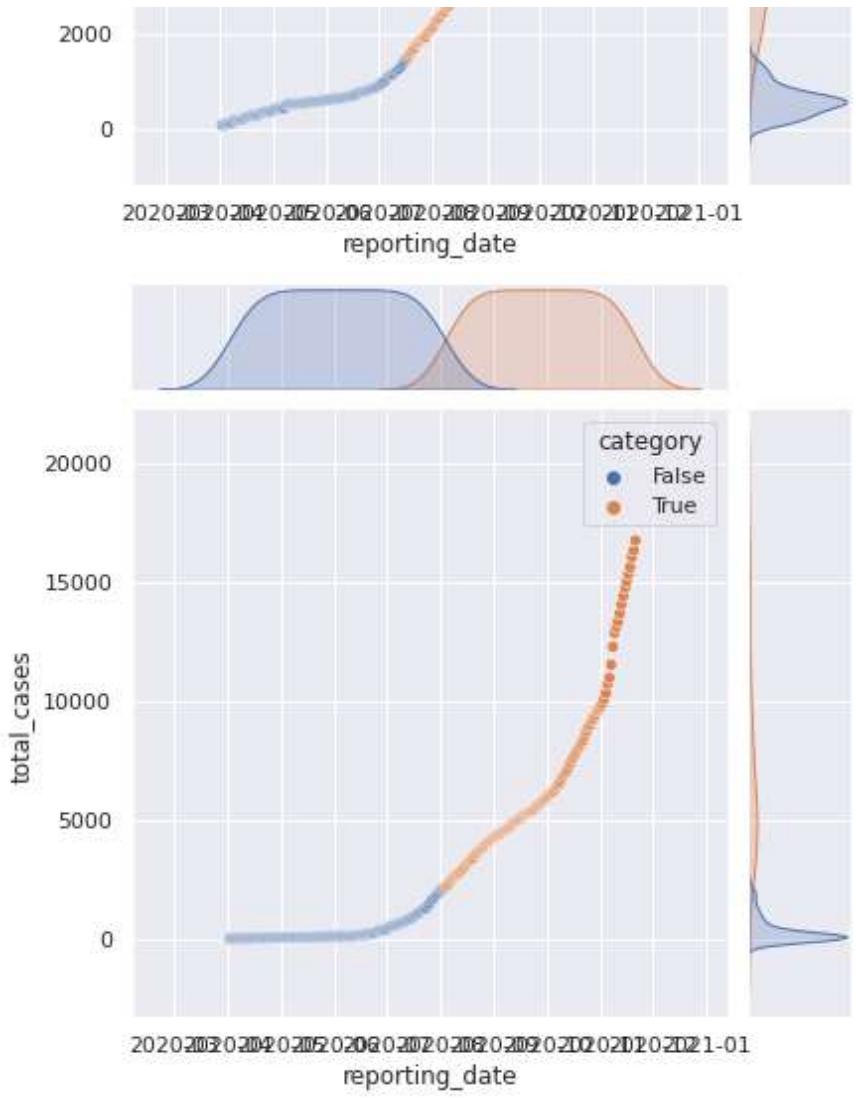
```
<seaborn.axisgrid.JointGrid at 0x7fdbab4114a8>
```










2000
0
reporting_date
category
False
True
20000
15000
10000
5000
0
total_cases
reporting_date